

AI ENGINEER INTERVIEW KIT

RAG, Agents & Production Failure Scenarios

A senior-level preparation guide for AI Engineer, GenAI Engineer, Forward Deployed AI Engineer, AI Solutions Engineer, and backend/platform engineers building production LLM systems.

By Tobias Weissmann

What this book trains you to do	Why it matters in interviews
Diagnose RAG hallucinations across ingestion, retrieval, context construction, generation, and evaluation.	Senior candidates do not blame the model first; they isolate failure modes through evidence.
Design reliable tool-using agents with guardrails, state, observability, and human approval.	Agent interviews test workflow judgment, not only prompt writing.
Answer production debugging questions with investigation paths, metrics, mitigations, and trade-offs.	Interviewers look for operational reasoning under uncertainty.
Use scorecards to compare weak, good, strong, and excellent answers.	Rubrics help candidates prepare like hiring panels evaluate.

Premium Interview Preparation Edition | Version 1.0 | 2026

How to Use This Book

This book is designed for candidates who already understand basic LLM terminology and want to perform well in senior, production-oriented AI engineering interviews. It focuses on failure modes, diagnosis, trade-offs, and communication. The strongest candidates do not simply describe RAG, agents, vector databases, or evaluation frameworks. They explain how those systems break and how they would prove what is actually broken.

Use this material in three passes. First, read Parts 1-3 to build the mental model for production debugging. Second, practice the agent design and production scenarios in Parts 4-6 out loud. Third, use the weak vs strong answer section and scorecards as a mock interview calibration tool.

This material is educational interview-preparation content. It is not legal, security, compliance, or professional consulting advice for any specific production deployment.

Manual Table of Contents

- Part 1 - How to Think Like a Production AI Engineer
- Part 2 - RAG Hallucination Failure Modes
- Part 3 - RAG Troubleshooting Checklist
- Part 4 - Agent System Design Questions
- Part 5 - Production Debugging Scenarios
- Part 6 - Evaluation Framework Questions
- Part 7 - Weak vs Strong Interview Answers
- Part 8 - Scorecards and Rubrics
- Part 9 - Final Interview Preparation Playbook
- Appendix - Reference Anchors and Cheat Sheets

Part 1 - How to Think Like a Production AI Engineer

A production AI engineer treats hallucination as a symptom, not a diagnosis. When an answer is wrong, the model may be the visible component that produced the final text, but the root cause may be upstream: a document that was never ingested, a parser that dropped a table, stale embeddings, a tenant filter bug, poor context ordering, a reranker regression, or an evaluation set that no longer represents real user questions.

In interviews, this distinction matters because many candidates can name the components of a GenAI stack but cannot reason through the failure chain. A senior candidate can say: 'I would not start by swapping the model. I would first reproduce the issue, inspect the retrieved evidence, trace the query through ingestion, retrieval, context construction, generation, and evaluation, then isolate whether the answer is unsupported, contradicted, stale, or inaccessible.'

1.1 Components vs Failure Modes

Component Knowledge	Failure-Mode Thinking
Knows that RAG uses embeddings, vector databases, prompts, and LLMs.	Asks whether the right documents were ingested, parsed, embedded, retrieved, ranked, filtered, and placed in the prompt.
Knows agents use tools and planning.	Asks whether the agent has correct tool permissions, retry behavior, state boundaries, idempotency, and escalation rules.
Knows evaluation uses metrics.	Separates retrieval recall, context precision, citation correctness, task success, latency, cost, and human satisfaction.
Suggests better prompts or a better model.	Builds an investigation tree and proves or disproves hypotheses with traces and metrics.

1.2 Why RAG Hallucinates Even When the LLM Is Strong

RAG improves factual grounding by giving the model external context, but it does not guarantee correctness. The LLM can only ground its answer in what it receives. If retrieval misses the relevant policy, if the context contains outdated material, if the prompt combines conflicting documents, or if the relevant sentence is buried in a long middle section, the model may still produce a fluent but unsupported answer.

- **Retrieval failure** - The answer exists in the corpus, but the retriever does not return it.
- **Context failure** - The correct chunk is retrieved but lost, truncated, badly ordered, or mixed with contradictory context.
- **Grounding failure** - The model ignores or over-generalizes from the supplied context.
- **Data freshness failure** - The answer was correct last week but the underlying source changed.
- **Evaluation failure** - Offline scores look healthy, but real user questions shifted outside the test distribution.

1.3 Why "Use a Better Model" Is Usually the Wrong First Answer

A stronger model can sometimes reduce hallucinations, but it is rarely the first diagnostic step. If the correct evidence is not in the prompt, even a stronger model may produce a confident answer without support. If tenant filters are wrong, a stronger model may make the privacy issue worse by reasoning more effectively over forbidden context. If an ingestion job failed, changing the model hides the operational bug instead of fixing it.

The senior answer is evidence-driven: reproduce the bad answer, capture the request trace, inspect retrieved chunks, compare against the source of truth, review ingestion freshness, check metadata and permissions, then decide whether model behavior is a root cause or only the last visible step.

1.4 The Production AI Debugging Loop

1. Reproduce the issue with the exact user query, tenant, timestamp, conversation history, and model configuration.
2. Classify the error: unsupported answer, wrong answer, stale answer, missing answer, wrong citation, policy violation, latency/cost regression, or tool/action failure.
3. Inspect retrieval: query rewrite, filters, top-k results, reranker scores, chunk versions, source metadata, and access-control constraints.
4. Inspect context construction: order, truncation, deduplication, token budget, conflicting evidence, citations, and compression.
5. Inspect generation: prompt version, model version, decoding settings, refusal rules, grounding instruction, and output parser.
6. Inspect evaluation and monitoring: golden set coverage, regression trend, user feedback, incident timeline, and recent deployments.
7. Mitigate safely: rollback, disable risky tools, reduce action autonomy, pin model/prompt versions, re-index documents, or force answer-with-citation behavior.
8. Fix permanently: add tests, observability, ingestion validation, retrieval evaluation, schema constraints, and release gates.

1.5 Weak vs Strong Thinking Example

Prompt	Weak answer	Strong answer
The RAG system worked last week. Now users get incorrect answers. Where do you investigate first?	I would try a better model or improve the prompt.	I would first compare a failing trace with a known-good trace. I would inspect retrieved chunks, document versions, metadata filters, ingestion status, embedding index freshness, prompt/context construction, and recent releases.
Users complain the answer cites a policy but the policy does not support the answer.	The citation feature may be buggy.	I would check whether citations are bound to answer spans or just attached from retrieved documents. Then I would inspect whether the model synthesized from one chunk while citing another, and whether citation validation exists.

Part 2 - RAG Hallucination Failure Modes

This section is the core production-debugging map. Each failure mode follows the same diagnostic structure: what it means, why it causes hallucination, symptoms, detection, debugging path, mitigation, and interview answer example.

1. Poor document ingestion

What it means: Documents are missing, partially loaded, duplicated, incorrectly routed, or stored under the wrong tenant, product, locale, or version.

Why it causes hallucination: If the source of truth never enters the retrieval corpus, the model must answer from incomplete or unrelated evidence.

Production symptoms: Recent documents are not reflected in answers; only older versions appear; some tenants or file types fail more often.

How to detect it: Compare source repository counts with indexed document counts; check ingestion job logs, dead-letter queues, parse error rates, and freshness timestamps.

How to debug it: Replay the ingestion event for a known failing document, verify raw file availability, parsed text, metadata, chunk creation, embedding creation, and index commit.

Fix or mitigation: Add ingestion completeness checks, checksums, retry queues, document version dashboards, and release gates that block serving stale or partial indexes.

Interview answer example: I would not assume the model forgot. I would verify whether the relevant document entered the index and whether its metadata allows it to be retrieved for this user.

2. Broken parsing or OCR

What it means: PDFs, scanned documents, tables, headers, footnotes, or images are converted into incomplete or corrupted text.

Why it causes hallucination: Retrieval may return chunks that look similar but omit the exact policy value, table cell, legal exception, or date needed for a grounded answer.

Production symptoms: Answers fail on PDFs, tables, images, invoices, contracts, or scanned policy documents; citations point to pages but text is missing.

How to detect it: Sample parsed output beside the original document; measure empty-page rate, OCR confidence, table extraction errors, and parser warnings.

How to debug it: Inspect the exact chunk shown to the model and compare it to the source page. Check whether table rows, units, dates, and headings survived parsing.

Fix or mitigation: Use layout-aware parsers, OCR confidence thresholds, human review for low-quality pages, table-specific chunking, and parser regression tests.

Interview answer example: If citations point to the right file but the answer is wrong, I would inspect whether the parsed text actually contains the information the model needed.

3. Bad chunking strategy

What it means: Chunks are too small, too large, split across semantic boundaries, or lose headings and surrounding context.

Why it causes hallucination: Small chunks lose meaning; large chunks dilute relevance; split tables or policies cause the model to combine incomplete evidence.

Production symptoms: Retrieved chunks contain fragments without definitions; answers miss exceptions; top results include nearby but incomplete text.

How to detect it: Review chunk length distribution, overlap, heading retention, chunk-level retrieval examples, and answer failures by document type.

How to debug it: Take a failing query, find the ideal source passage manually, then check whether it exists as one coherent retrievable chunk.

Fix or mitigation: Use heading-aware, semantic, parent-child, table-aware, or code-aware chunking; preserve section hierarchy and source metadata.

Interview answer example: I would ask whether the relevant fact is retrievable as a complete unit. Chunking is not just token splitting; it is evidence design.

4. Missing metadata

What it means: Chunks lack product, tenant, region, date, language, permission, author, version, or document-type metadata.

Why it causes hallucination: The retriever cannot filter to the correct scope and may return plausible but wrong context from another product, region, or time period.

Production symptoms: Answers are often correct in general but wrong for specific customers, geographies, or versions.

How to detect it: Audit metadata coverage and null rates; compare expected metadata from source systems with indexed metadata.

How to debug it: Inspect failing chunks and ask: would the system know whether this evidence belongs to the user, region, language, and current version?

Fix or mitigation: Make metadata mandatory, validate at ingestion, backfill indexes, and add metadata-aware evaluation cases.

Interview answer example: If the issue depends on region or customer, I would suspect missing or wrong metadata before blaming the LLM.

5. Incorrect metadata filters

What it means: Filters are too broad, too narrow, applied after retrieval instead of before, or use the wrong field/value mapping.

Why it causes hallucination: Relevant evidence may be filtered out, or forbidden/irrelevant evidence may enter the context.

Production symptoms: Some users get no answer; others get answers from the wrong tenant; retrieval looks good in dev but fails in production.

How to detect it: Log the final filter expression; test exact tenant/product/date/language values; compare pre-filter vs post-filter retrieval.

How to debug it: Run the failing query with and without filters in a safe test environment and inspect which document disappears or appears.

Fix or mitigation: Centralize filter construction, add unit tests for ACL and metadata filters, and add canary queries for each tenant and region.

Interview answer example: I would inspect the final filter actually sent to the vector store, not only the intended filter in the code.

6. Embedding model mismatch

What it means: Documents and queries are embedded with different models, dimensions, normalization rules, or preprocessing steps.

Why it causes hallucination: Similarity scores become unreliable, so retrieval returns semantically weak or random matches.

Production symptoms: Quality drops after an embedding migration; many scores cluster oddly; multilingual or domain terms regress.

How to detect it: Check embedding model IDs, dimensions, preprocessing, normalization, and index namespaces; compare score distributions before and after change.

How to debug it: Embed a known query and known relevant chunk with the serving path, then verify they are in the same vector space.

Fix or mitigation: Version embeddings explicitly, rebuild indexes atomically, and run retrieval regression before switching traffic.

Interview answer example: If retrieval quality suddenly dropped after a deployment, embedding/index compatibility is one of my first checks.

7. Stale embeddings

What it means: The document text changed but the stored vectors still represent the old content.

Why it causes hallucination: The retriever finds chunks based on outdated meaning, and the model answers from stale evidence.

Production symptoms: Source page shows new policy but answer reflects old policy; chunk text or embeddings have older timestamps than the document.

How to detect it: Compare document modified time, parse time, embedding time, and index commit time.

How to debug it: Find the source document version and verify the chunk text and embedding version used at answer time.

Fix or mitigation: Use incremental re-embedding queues, invalidation on document updates, and freshness SLOs.

Interview answer example: I would inspect whether the answer is wrong because the text is stale or because the retriever is stale.

8. Vector index not updated

What it means: New embeddings exist but were not committed, synced, replicated, or routed to the serving index.

Why it causes hallucination: Serving still retrieves old chunks even after ingestion completed.

Production symptoms: Back-office says documents are uploaded; production answers ignore them; staging may work while production fails.

How to detect it: Check index build status, replica sync, namespace routing, alias version, and cache invalidation.

How to debug it: Query the serving index directly for the new document ID and compare with the ingestion index.

Fix or mitigation: Use blue/green indexes, atomic alias swaps, index health checks, and document-count parity checks.

Interview answer example: Completed ingestion is not enough. I would verify the serving index contains and can retrieve the new chunk.

9. Retrieval misses

What it means: The correct evidence exists but is not returned in top-k results.

Why it causes hallucination: The model never sees the right source and fills gaps with plausible text.

Production symptoms: Manual search finds the answer; RAG response does not; top-k results are semantically adjacent but not sufficient.

How to detect it: Use labeled queries and measure recall@k; inspect nearest neighbors and query rewrite output.

How to debug it: Run lexical, semantic, hybrid, and manual searches to find where recall fails.

Fix or mitigation: Improve query expansion, hybrid search, reranking, metadata, chunking, and domain synonym handling.

Interview answer example: I would measure retrieval recall separately from answer quality so I can prove whether retrieval is the bottleneck.

10. Low recall retrieval

What it means: The retriever returns too few candidate documents or uses thresholds that remove relevant but lower-scoring evidence.

Why it causes hallucination: The final context lacks complete evidence, especially for multi-hop or rare queries.

Production symptoms: The system answers common queries but fails on edge cases, long-tail terminology, or multi-document questions.

How to detect it: Track recall@k, no-result rate, threshold drop rate, and query classes with low coverage.

How to debug it: Increase candidate pool in offline tests and see whether the answer becomes available before reranking.

Fix or mitigation: Tune top-k, candidate generation, hybrid weighting, synonym expansion, and reranker depth.

Interview answer example: A low top-k can make the model look bad even when the corpus contains the answer.

11. High similarity but wrong context

What it means: Retriever returns chunks with similar vocabulary but different intent, product, or policy scope.

Why it causes hallucination: The model grounds on superficially similar evidence and produces a confident but wrong answer.

Production symptoms: Top chunks look relevant by keywords but answer the wrong question; score is high but semantic fit is weak.

How to detect it: Review false positives, context precision, reranker judgments, and query/document intent labels.

How to debug it: Compare ideal answer evidence with retrieved evidence and identify terms causing false similarity.

Fix or mitigation: Add metadata filters, reranking, hard negatives, better chunk titles, and intent-aware retrieval.

Interview answer example: Similarity is not truth. I would inspect whether the chunk answers the specific user intent.

12. Query rewriting failure

What it means: The system rewrites the user query in a way that changes intent, removes constraints, or over-expands synonyms.

Why it causes hallucination: Retrieval searches for a cleaner but wrong question, so the context does not match the original request.

Production symptoms: Raw query search works; rewritten query fails; multi-turn questions lose pronouns, dates, product names, or negations.

How to detect it: Log raw query, rewritten query, conversation summary, and retrieval results for both.

How to debug it: Replay retrieval with raw and rewritten queries. Check whether the rewrite dropped the critical constraint.

Fix or mitigation: Constrain rewrites, preserve entities and negations, evaluate rewrites, and fall back to raw query retrieval.

Interview answer example: I would treat query rewriting as a model component with its own regression tests.

13. Hybrid search misconfiguration

What it means: Dense and lexical search are combined with poor weighting, normalization, or result merging.

Why it causes hallucination: Important exact-match terms such as product codes, legal clauses, error codes, or acronyms may be underweighted.

Production symptoms: Semantic search works for broad concepts but fails for IDs, dates, SKUs, names, or short technical queries.

How to detect it: Compare dense-only, lexical-only, and hybrid retrieval; inspect score normalization and merge strategy.

How to debug it: Run test queries that require exact matching and check whether lexical candidates survive reranking.

Fix or mitigation: Tune hybrid weights per query class, protect exact matches, and add query intent routing.

Interview answer example: For enterprise RAG, I would not rely only on vector similarity; exact terms often carry business meaning.

14. Reranker failure

What it means: A reranker demotes the correct chunk or promotes attractive but insufficient chunks.

Why it causes hallucination: The candidate set may contain the answer, but the final context excludes it.

Production symptoms: Top-50 has the answer; top-5 sent to the model does not; quality drops after adding reranking.

How to detect it: Log candidate ranks before and after reranking; evaluate MRR, nDCG, and answer coverage.

How to debug it: Inspect failing query pairs and verify whether the reranker understands domain terms and metadata.

Fix or mitigation: Fine-tune or replace the reranker, add metadata features, increase context diversity, and add reranker regression tests.

Interview answer example: I would check not only what retrieval found but what the reranker removed.

15. Context window overflow

What it means: Too much retrieved content exceeds the model context budget, causing truncation or compression.

Why it causes hallucination: Relevant evidence may be cut off while irrelevant or earlier content remains.

Production symptoms: Answers fail on broad or multi-document queries; traces show dropped chunks or aggressive compression.

How to detect it: Track token budget, truncation events, chunk inclusion rate, and answer quality by context length.

How to debug it: Inspect the final prompt, not only retrieval results. Verify whether the answer evidence was actually included.

Fix or mitigation: Use context budgeting, summarization with validation, parent-child retrieval, and evidence prioritization.

Interview answer example: Retrieved is not the same as seen. I would inspect the final model input.

16. Poor context ordering

What it means: Chunks are ordered by score, time, source, or arbitrary merge logic in a way that hides the most useful evidence.

Why it causes hallucination: The model may over-weight early or late context and ignore a critical middle chunk.

Production symptoms: Correct chunk is present but answer follows another chunk; quality changes when chunk order changes.

How to detect it: Log final context order and test order sensitivity on known cases.

How to debug it: Move the gold evidence to different positions and see whether the answer changes.

Fix or mitigation: Place critical evidence near instructions, group by source, remove duplicates, and cite evidence at span level.

Interview answer example: Context construction is part of model behavior. Ordering can change the answer.

17. Lost-in-the-middle problem

What it means: Relevant evidence appears in the middle of a long context where the model uses it less reliably.

Why it causes hallucination: The model may answer from easier-to-access beginning/end context rather than the decisive middle evidence.

Production symptoms: Answer improves when the same chunk is moved earlier; long-context models still miss middle evidence.

How to detect it: Run position-sensitivity tests and compare answer accuracy by evidence position.

How to debug it: Create controlled prompts with the same evidence at different positions.

Fix or mitigation: Reduce context size, put critical evidence near the front, summarize with verification, and use retrieval diversity rather than stuffing.

Interview answer example: Long context is not a substitute for good retrieval and context design.

18. Prompt/context mismatch

What it means: The prompt asks the model to behave in a way that conflicts with retrieved evidence or system constraints.

Why it causes hallucination: The model may prioritize style, completeness, or helpfulness over strict grounding.

Production symptoms: Answers are confident but not cited; refusal behavior is inconsistent; model over-explains beyond context.

How to detect it: Review prompt versions, grounding rules, few-shot examples, and output schema.

How to debug it: Compare the prompt contract with the actual answer. Check whether examples encourage unsupported synthesis.

Fix or mitigation: Use explicit grounding rules, answerability checks, citation validation, and output schemas.

Interview answer example: I would check whether the prompt rewards useful-looking answers more than faithful answers.

19. Conflicting documents

What it means: Multiple retrieved documents contain different versions, exceptions, or contradictory statements.

Why it causes hallucination: The model blends them or chooses the wrong one without acknowledging conflict.

Production symptoms: Answers vary by retrieval order; citations include old and new policies; users report inconsistent answers.

How to detect it: Detect conflicting evidence, version metadata, source authority, and effective dates.

How to debug it: Inspect all retrieved documents and ask which one should win according to business rules.

Fix or mitigation: Use source authority ranking, effective-date filters, conflict detection, and explicit uncertainty responses.

Interview answer example: When evidence conflicts, a strong system should surface the conflict or apply authority rules, not hallucinate a compromise.

20. Stale documents

What it means: Documents are correctly indexed but no longer valid for the current answer.

Why it causes hallucination: The model faithfully answers from outdated evidence, producing a wrong business answer.

Production symptoms: Citations are real but old; answers fail around policy changes, releases, or legal updates.

How to detect it: Track document effective dates, expiry dates, version status, and freshness metrics.

How to debug it: Compare retrieved source dates against current source-of-truth rules.

Fix or mitigation: Use lifecycle metadata, archival filters, freshness alerts, and answer disclaimers for stale sources.

Interview answer example: A grounded answer can still be wrong if grounded in stale evidence.

21. Permission leakage or missing access filters

What it means: Retrieval includes documents the user is not allowed to see, or excludes documents they should access.

Why it causes hallucination: The model may expose sensitive data or answer incorrectly because relevant permitted data is absent.

Production symptoms: Cross-tenant leakage, customer-specific answers, or suspicious citations from restricted sources.

How to detect it: Audit ACL filters, tenant IDs, group membership, row-level permissions, and retrieval traces.

How to debug it: Test with boundary users and compare expected vs actual accessible documents.

Fix or mitigation: Enforce permissions before retrieval, not only after generation; add security regression tests and deny-by-default behavior.

Interview answer example: Security filters are part of retrieval correctness. I would validate them before debugging the prompt.

22. Summarization compression errors

What it means: Retrieved chunks are summarized before generation and the summary drops qualifiers, exceptions, dates, or uncertainty.

Why it causes hallucination: The model answers from a compressed representation that is easier to read but less faithful.

Production symptoms: Detailed source has caveats; answer lacks caveats; summary logs differ from original chunks.

How to detect it: Compare original chunks, compressed context, and final answer; measure compression faithfulness.

How to debug it: Run the failing query without compression or with extractive compression to isolate the issue.

Fix or mitigation: Use extractive summaries, preserve citations, verify summaries against source, and avoid compressing legal or numeric evidence.

Interview answer example: Compression is another generative step that can hallucinate before the final answer even begins.

23. Multi-hop retrieval failure

What it means: The answer requires evidence from multiple documents or sections, but retrieval only finds one hop.

Why it causes hallucination: The model fills the missing hop with assumptions.

Production symptoms: Questions involving eligibility, exceptions, dependencies, or workflows are often wrong.

How to detect it: Evaluate multi-hop test sets and measure whether all required evidence pieces are retrieved.

How to debug it: Identify required evidence chain manually and compare against retrieval results.

Fix or mitigation: Use iterative retrieval, decomposition, graph retrieval, parent-child retrieval, and explicit evidence completeness checks.

Interview answer example: For multi-hop questions, I would ask whether the context contains every required fact, not merely one relevant chunk.

24. Evaluation dataset drift

What it means: Offline test queries no longer represent real user questions, documents, or workflows.

Why it causes hallucination: Metrics remain stable while production quality degrades.

Production symptoms: Dashboards look green; support tickets rise; new product areas or customer segments fail.

How to detect it: Compare production query logs, feedback, and ticket categories against the golden set.

How to debug it: Cluster recent failures and identify missing query classes in evaluation.

Fix or mitigation: Refresh golden datasets, add production-derived tests, and segment metrics by domain, language, tenant, and document type.

Interview answer example: Good evaluation is a moving target. I would inspect whether our tests still represent production usage.

25. No observability across the RAG pipeline

What it means: The system lacks traces connecting query, retrieval, context, model input, output, citations, cost, latency, and feedback.

Why it causes hallucination: Teams cannot identify root cause and default to guessing or model swapping.

Production symptoms: Incidents take long to debug; only final answer logs exist; no visibility into retrieved evidence.

How to detect it: Check whether traces include query rewrite, filters, retrieved chunk IDs, scores, context tokens, prompt version, model version, and feedback.

How to debug it: Reproduce a failure and see whether every pipeline stage is observable.

Fix or mitigation: Add end-to-end tracing, structured logs, span-level metrics, retrieval snapshots, and evaluation dashboards.

Interview answer example: Without observability, RAG debugging becomes opinion-based. I would build traces before making major changes.

Part 3 - RAG Troubleshooting Checklist

Use this checklist when an interviewer gives you a vague production symptom such as: "The RAG system started hallucinating" or "Users say answers are wrong." Your answer should move stage by stage through the pipeline and gather evidence before jumping to a solution.

User Query Layer

Query ambiguity

What to inspect: Whether query ambiguity is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how query ambiguity could create a wrong answer and names the artifact or metric they would inspect first.

Query rewriting

What to inspect: Whether query rewriting is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how query rewriting could create a wrong answer and names the artifact or metric they would inspect first.

Intent detection

What to inspect: Whether intent detection is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how intent detection could create a wrong answer and names the artifact or metric they would inspect first.

Language mismatch

What to inspect: Whether language mismatch is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how language mismatch could create a wrong answer and names the artifact or metric they would inspect first.

Multi-turn conversation issues

What to inspect: Whether multi-turn conversation issues is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how multi-turn conversation issues could create a wrong answer and names the artifact or metric they would inspect first.

Ingestion Layer

File loading

What to inspect: Whether file loading is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how file loading could create a wrong answer and names the artifact or metric they would inspect first.

Parsing

What to inspect: Whether parsing is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how parsing could create a wrong answer and names the artifact or metric they would inspect first.

OCR

What to inspect: Whether ocr is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how ocr could create a wrong answer and names the artifact or metric they would inspect first.

Table extraction

What to inspect: Whether table extraction is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how table extraction could create a wrong answer and names the artifact or metric they would inspect first.

Image/document handling

What to inspect: Whether image/document handling is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how image/document handling could create a wrong answer and names the artifact or metric they would inspect first.

Deduplication

What to inspect: Whether deduplication is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how deduplication could create a wrong answer and names the artifact or metric they would inspect first.

Document versioning

What to inspect: Whether document versioning is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how document versioning could create a wrong answer and names the artifact or metric they would inspect first.

Chunking Layer

Chunk size

What to inspect: Whether chunk size is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how chunk size could create a wrong answer and names the artifact or metric they would inspect first.

Chunk overlap

What to inspect: Whether chunk overlap is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how chunk overlap could create a wrong answer and names the artifact or metric they would inspect first.

Semantic chunking

What to inspect: Whether semantic chunking is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how semantic chunking could create a wrong answer and names the artifact or metric they would inspect first.

Header-aware chunking

What to inspect: Whether header-aware chunking is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how header-aware chunking could create a wrong answer and names the artifact or metric they would inspect first.

Table/code chunking

What to inspect: Whether table/code chunking is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how table/code chunking could create a wrong answer and names the artifact or metric they would inspect first.

Parent-child chunking

What to inspect: Whether parent-child chunking is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how parent-child chunking could create a wrong answer and names the artifact or metric they would inspect first.

Embedding Layer

Embedding model choice

What to inspect: Whether embedding model choice is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how embedding model choice could create a wrong answer and names the artifact or metric they would inspect first.

Embedding drift

What to inspect: Whether embedding drift is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how embedding drift could create a wrong answer and names the artifact or metric they would inspect first.

Re-embedding strategy

What to inspect: Whether re-embedding strategy is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how re-embedding strategy could create a wrong answer and names the artifact or metric they would inspect first.

Domain-specific terminology

What to inspect: Whether domain-specific terminology is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how domain-specific terminology could create a wrong answer and names the artifact or metric they would inspect first.

Multilingual embeddings

What to inspect: Whether multilingual embeddings is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how multilingual embeddings could create a wrong answer and names the artifact or metric they would inspect first.

Retrieval Layer

Top-k

What to inspect: Whether top-k is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how top-k could create a wrong answer and names the artifact or metric they would inspect first.

Similarity thresholds

What to inspect: Whether similarity thresholds is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how similarity thresholds could create a wrong answer and names the artifact or metric they would inspect first.

Hybrid search

What to inspect: Whether hybrid search is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how hybrid search could create a wrong answer and names the artifact or metric they would inspect first.

Metadata filters

What to inspect: Whether metadata filters is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how metadata filters could create a wrong answer and names the artifact or metric they would inspect first.

Reranking

What to inspect: Whether reranking is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how reranking could create a wrong answer and names the artifact or metric they would inspect first.

Recall vs precision

What to inspect: Whether recall vs precision is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how recall vs precision could create a wrong answer and names the artifact or metric they would inspect first.

Access control filters

What to inspect: Whether access control filters is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how access control filters could create a wrong answer and names the artifact or metric they would inspect first.

Context Construction Layer

Context ordering

What to inspect: Whether context ordering is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how context ordering could create a wrong answer and names the artifact or metric they would inspect first.

Deduplication

What to inspect: Whether deduplication is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how deduplication could create a wrong answer and names the artifact or metric they would inspect first.

Token budget

What to inspect: Whether token budget is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how token budget could create a wrong answer and names the artifact or metric they would inspect first.

Citations

What to inspect: Whether citations is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how citations could create a wrong answer and names the artifact or metric they would inspect first.

Conflicting context

What to inspect: Whether conflicting context is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how conflicting context could create a wrong answer and names the artifact or metric they would inspect first.

Context compression

What to inspect: Whether context compression is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how context compression could create a wrong answer and names the artifact or metric they would inspect first.

Generation Layer

Prompt structure

What to inspect: Whether prompt structure is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how prompt structure could create a wrong answer and names the artifact or metric they would inspect first.

Grounding instructions

What to inspect: Whether grounding instructions is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how grounding instructions could create a wrong answer and names the artifact or metric they would inspect first.

Refusal behavior

What to inspect: Whether refusal behavior is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how refusal behavior could create a wrong answer and names the artifact or metric they would inspect first.

Citation requirements

What to inspect: Whether citation requirements is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how citation requirements could create a wrong answer and names the artifact or metric they would inspect first.

Temperature and decoding settings

What to inspect: Whether temperature and decoding settings is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how temperature and decoding settings could create a wrong answer and names the artifact or metric they would inspect first.

Evaluation Layer

Golden datasets

What to inspect: Whether golden datasets is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how golden datasets could create a wrong answer and names the artifact or metric they would inspect first.

Regression tests

What to inspect: Whether regression tests is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how regression tests could create a wrong answer and names the artifact or metric they would inspect first.

Faithfulness metrics

What to inspect: Whether faithfulness metrics is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how faithfulness metrics could create a wrong answer and names the artifact or metric they would inspect first.

Answer relevance

What to inspect: Whether answer relevance is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how answer relevance could create a wrong answer and names the artifact or metric they would inspect first.

Retrieval recall

What to inspect: Whether retrieval recall is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how retrieval recall could create a wrong answer and names the artifact or metric they would inspect first.

Human review

What to inspect: Whether human review is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how human review could create a wrong answer and names the artifact or metric they would inspect first.

Online feedback loops

What to inspect: Whether online feedback loops is preserving user intent, document scope, permissions, and the required evidence path.

Logs/metrics: Trace logs, retrieval examples, error clusters, score distributions, latency/cost, feedback labels, and before/after regression cases.

Weak candidate: Blames the prompt, model, or temperature without checking this layer.

Strong candidate: Explains how online feedback loops could create a wrong answer and names the artifact or metric they would inspect first.

Part 4 - Agent System Design Questions

Agent interviews test whether a candidate can design controlled autonomy. The goal is not to say "the agent will reason and call tools." The goal is to define tool contracts, state boundaries, permission models, human approval points, observability, recovery behavior, and evaluation criteria.

1. Design a customer-support agent that can answer policy questions and create support tickets.

What the interviewer is testing: tool selection, grounding, escalation, tenant safety, ticket idempotency

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses tool selection, grounding, escalation, tenant safety, ticket idempotency.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

2. Design an agent that helps an enterprise sales team prepare account briefings from CRM, email, and knowledge-base data.

What the interviewer is testing: multi-source retrieval, privacy, summarization, evidence quality, action boundaries

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses multi-source retrieval, privacy, summarization, evidence quality, action boundaries.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

3. Design a coding assistant agent that can read a repository, suggest fixes, and open a pull request.

What the interviewer is testing: repository access, sandboxing, tests, diff generation, human approval

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses repository access, sandboxing, tests, diff generation, human approval.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

4. Design an agentic workflow for invoice dispute resolution.

What the interviewer is testing: structured workflow, document extraction, approvals, audit trails

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses structured workflow, document extraction, approvals, audit trails.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

5. Design a research agent that collects sources, summarizes findings, and cites evidence.

What the interviewer is testing: source credibility, retrieval loops, citation integrity, uncertainty

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses source credibility, retrieval loops, citation integrity, uncertainty.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

6. Design an agent for IT helpdesk automation that can reset passwords and create incidents.

What the interviewer is testing: security permissions, sensitive actions, human-in-loop

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses security permissions, sensitive actions, human-in-loop.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

7. Design a multi-step travel-policy compliance agent for employee expenses.

What the interviewer is testing: policy retrieval, tool calls, edge cases, explanations

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses policy retrieval, tool calls, edge cases, explanations.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

8. Design an agent that monitors production logs and proposes remediation steps.

What the interviewer is testing: observability, false positives, safety, runbook integration

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses observability, false positives, safety, runbook integration.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

9. Design an agent for onboarding new enterprise customers onto a SaaS platform.

What the interviewer is testing: workflow orchestration, data collection, APIs, checkpoints

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses workflow orchestration, data collection, APIs, checkpoints.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

10. Design a legal contract review assistant for internal counsel.

What the interviewer is testing: confidentiality, citations, risk classification, non-authoritative output

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses confidentiality, citations, risk classification, non-authoritative output.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

11. Design an agent that updates CRM records after sales calls.

What the interviewer is testing: transcription quality, structured extraction, approval, rollback

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses transcription quality, structured extraction, approval, rollback.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

12. Design an agent that plans and executes data migration tasks.

What the interviewer is testing: long-running state, checkpoints, idempotency, rollback

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses long-running state, checkpoints, idempotency, rollback.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

13. Design an agent that books meetings across multiple calendars and time zones.

What the interviewer is testing: tool failure, constraints, user preferences, confirmation

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses tool failure, constraints, user preferences, confirmation.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

14. Design an agent for financial-report Q&A over internal documents.

What the interviewer is testing: freshness, permissions, numeric accuracy, citations

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses freshness, permissions, numeric accuracy, citations.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

15. Design an agent that can call external APIs with rate limits and partial failures.

What the interviewer is testing: retries, backoff, circuit breakers, cost control

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses retries, backoff, circuit breakers, cost control.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

16. Design an agent memory system for recurring customer interactions.

What the interviewer is testing: memory scope, privacy, expiration, personalization boundaries

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses memory scope, privacy, expiration, personalization boundaries.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

17. Design a human approval layer for high-risk agent actions.

What the interviewer is testing: risk classification, policy gates, escalation, auditability

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses risk classification, policy gates, escalation, auditability.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

18. Design an agent evaluation framework for tool-using tasks.

What the interviewer is testing: task success, tool accuracy, groundedness, cost, latency

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses task success, tool accuracy, groundedness, cost, latency.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

19. Design a multi-agent system for complex enterprise workflows.

What the interviewer is testing: coordination, communication, state ownership, failure isolation

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses coordination, communication, state ownership, failure isolation.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

20. Design an agent that detects and resists prompt injection in documents and user prompts.

What the interviewer is testing: security boundaries, content isolation, tool restrictions

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses security boundaries, content isolation, tool restrictions.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

21. Design an agent for multilingual customer support.

What the interviewer is testing: language detection, multilingual retrieval, localization, escalation

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses language detection, multilingual retrieval, localization, escalation.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

22. Design an agent that generates SQL queries from business questions.

What the interviewer is testing: schema grounding, validation, read-only safety, result interpretation

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses schema grounding, validation, read-only safety, result interpretation.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

23. Design an agent for Kubernetes incident triage.

What the interviewer is testing: observability integrations, safe commands, runbooks, approval

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses observability integrations, safe commands, runbooks, approval.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

24. Design an agent that processes long-running procurement workflows.

What the interviewer is testing: state persistence, retries, human checkpoints, SLA tracking

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses state persistence, retries, human checkpoints, SLA tracking.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

25. Design a cost-aware agent platform for thousands of users.

What the interviewer is testing: routing, caching, budgets, model selection, monitoring

Strong answer outline: A strong answer defines workflow boundaries, tool contracts, read vs write actions, permission checks, observability, evaluation, fallback behavior, and human approval for risky steps. It directly addresses routing, caching, budgets, model selection, monitoring.

Common weak answer: A weak answer says the agent will reason, use tools, and solve the problem without defining state, permissions, failure handling, or evaluation.

Follow-up questions: How do you prevent wrong tool calls? What happens if a tool times out? Which actions require approval? How do you evaluate task success? What do you log for incident debugging?

Scorecard/rubric: 5/5: defines safe autonomy, evidence, state, tools, evaluation, and failure modes. 3/5: covers basic architecture but misses operational controls. 1/5: treats the agent as magic prompt orchestration.

Part 5 - Production Debugging Scenarios

Practice these scenarios out loud. For each one, force yourself to name likely root causes, investigation path, logs/metrics, short-term mitigation, long-term fix, and a strong interview answer. The goal is not one perfect answer; the goal is to show structured diagnosis.

1. RAG system worked last week, now answers are wrong

Situation: Production users report: rag system worked last week, now answers are wrong.

Likely root causes: Regression after document, index, prompt, model, filter, or deployment change.

Investigation path: Capture failing traces, compare against last known good, inspect retrieved chunks, document timestamps, index aliases, prompt/model versions, and recent deployments.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

2. Users report hallucinated policy answers

Situation: Production users report: users report hallucinated policy answers.

Likely root causes: Missing/stale policy, bad chunking, conflicting documents, weak grounding.

Investigation path: Inspect cited source, source authority, effective dates, answer support, and whether refusal was expected when context was insufficient.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

3. Retrieval returns irrelevant chunks

Situation: Production users report: retrieval returns irrelevant chunks.

Likely root causes: Embedding mismatch, bad query rewrite, missing metadata, poor hybrid tuning.

Investigation path: Compare raw vs rewritten query, dense vs lexical results, filters, score distributions, and known-gold retrieval examples.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

4. New documents were uploaded but answers still use old documents

Situation: Production users report: new documents were uploaded but answers still use old documents.

Likely root causes: Ingestion, embedding, index commit, cache, or alias sync failure.

Investigation path: Trace document lifecycle from source upload to parser, chunker, embedder, index, serving namespace, and cache invalidation.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

5. Agent calls the wrong tool

Situation: Production users report: agent calls the wrong tool.

Likely root causes: Ambiguous tool descriptions, missing routing rules, poor tool schema, weak state.

Investigation path: Inspect tool selection trace, tool descriptions, examples, arguments, planner output, and whether risk policy blocked the call.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

6. Agent loops forever

Situation: Production users report: agent loops forever.

Likely root causes: No termination criteria, retry bug, tool result not recognized, planner state corruption.

Investigation path: Check iteration count, state transitions, repeated observations, retry policy, and task completion detector.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

7. Agent gives correct answer but cites wrong source

Situation: Production users report: agent gives correct answer but cites wrong source.

Likely root causes: Citation binding is document-level not span-level, context mixing, post-processing bug.

Investigation path: Inspect answer spans, retrieved chunks, citation mapper, and whether cited text supports the exact claim.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

8. RAG works in English but fails in German

Situation: Production users report: rag works in english but fails in german.

Likely root causes: Embedding model or chunking not multilingual, query rewrite loses meaning, German docs parsed poorly.

Investigation path: Compare multilingual retrieval recall, language detection, tokenizer behavior, German synonyms, and source parsing quality.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

9. Embeddings regenerated and quality dropped

Situation: Production users report: embeddings regenerated and quality dropped.

Likely root causes: New model mismatch, changed preprocessing, index namespace conflict, poor migration.

Investigation path: Compare pre/post score distributions, recall@k, sample nearest neighbors, model IDs, normalization, and vector dimensions.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

10. Latency doubled after adding reranking

Situation: Production users report: latency doubled after adding reranking.

Likely root causes: Reranker depth too high, model too slow, no caching, batch inefficiency.

Investigation path: Break down latency by retrieval, reranking, context building, model generation; tune candidate count and cache hot queries.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

11. Prompt injection caused restricted information disclosure

Situation: Production users report: prompt injection caused restricted information disclosure.

Likely root causes: Untrusted content was treated as instruction, tools had excessive permissions, missing output validation.

Investigation path: Separate data from instructions, audit tool access, apply allowlists, add prompt-injection tests, and require approval for sensitive actions.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

12. Evaluation metrics look good but users are unhappy

Situation: Production users report: evaluation metrics look good but users are unhappy.

Likely root causes: Golden set drift, wrong metric, missing segments, UX problem, citation issue.

Investigation path: Cluster user complaints, compare with test coverage, segment by tenant/language/task, and add human evaluation.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

13. Customer-specific answers leak across tenants

Situation: Production users report: customer-specific answers leak across tenants.

Likely root causes: ACL filter bug, shared namespace, cache key missing tenant, post-retrieval filtering.

Investigation path: Inspect retrieval filters, cache keys, tenant metadata, access-control tests, and deny-by-default behavior.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

14. Long documents produce incomplete answers

Situation: Production users report: long documents produce incomplete answers.

Likely root causes: Chunking, context budget, lost-in-middle, table parsing, multi-hop failure.

Investigation path: Check chunk structure, parent-child retrieval, context truncation, evidence position, and whether all required sections were retrieved.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

15. Tables are not interpreted correctly

Situation: Production users report: tables are not interpreted correctly.

Likely root causes: Parser lost rows/columns, chunking split table, model misread units.

Investigation path: Compare original table to extracted text, use table-aware parsing, preserve headers, and test numeric/cell-level questions.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

16. RAG says it cannot answer despite documents containing answer

Situation: Production users report: rag says it cannot answer despite documents containing answer.

Likely root causes: Threshold too strict, filters too narrow, query rewrite mismatch, refusal over-trigger.

Investigation path: Test with raw query, wider top-k, disabled filters in safe mode, and inspect answerability classifier.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

17. Answers become too verbose and include unsupported advice

Situation: Production users report: answers become too verbose and include unsupported advice.

Likely root causes: Prompt change, missing grounding instruction, temperature, style examples.

Investigation path: Compare prompt version and few-shot examples; add answer-with-evidence constraints and unsupported-claim checks.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

18. Agent performs duplicate actions

Situation: Production users report: agent performs duplicate actions.

Likely root causes: No idempotency key, retry after timeout, state not persisted, tool response lost.

Investigation path: Check action logs, request IDs, retry policy, transaction boundaries, and implement idempotent tool calls.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

19. Tool API changed and agent quality dropped

Situation: Production users report: tool api changed and agent quality dropped.

Likely root causes: Schema drift, validation missing, planner still uses old fields.

Investigation path: Inspect tool errors, argument schemas, API versioning, contract tests, and fallback behavior.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

20. RAG quality is good offline but poor in production

Situation: Production users report: rag quality is good offline but poor in production.

Likely root causes: Test queries not representative, production query ambiguity, tenant/language drift.

Investigation path: Compare production logs with golden set, segment failures, and update evaluation with recent real-world cases.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

21. Model upgrade reduced answer faithfulness

Situation: Production users report: model upgrade reduced answer faithfulness.

Likely root causes: New model follows style better but grounding worse, prompt incompatible.

Investigation path: Run regression tests by claim support, citation accuracy, refusal behavior, latency, and cost before rollout.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

22. Users ask multi-turn questions and answers lose context

Situation: Production users report: users ask multi-turn questions and answers lose context.

Likely root causes: Conversation summarizer drops constraints, coreference failure, wrong history window.

Investigation path: Log conversation state, rewritten query, memory summary, and test pronoun/date/entity preservation.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

23. Agent selects expensive model for simple tasks

Situation: Production users report: agent selects expensive model for simple tasks.

Likely root causes: No routing policy, missing cost budget, planner overuses high-capability model.

Investigation path: Add model routing, budget controls, cache, quality gates, and cost per successful task metrics.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

24. RAG answer is correct but customer does not trust it

Situation: Production users report: rag answer is correct but customer does not trust it.

Likely root causes: Poor citations, weak explanation, no confidence handling, UX issue.

Investigation path: Improve citation snippets, answer structure, source authority display, and uncertainty communication.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

25. Retrieval returns correct document but wrong section

Situation: Production users report: retrieval returns correct document but wrong section.

Likely root causes: Chunk titles missing, large document, lexical collision, no section hierarchy.

Investigation path: Use section-aware metadata, parent-child retrieval, and reranking based on section intent.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

26. System fails after moving from staging to production

Situation: Production users report: system fails after moving from staging to production.

Likely root causes: Different index, secrets, permissions, data volume, model version, or environment config.

Investigation path: Compare config, model/prompt versions, index aliases, metadata filters, and production data characteristics.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

27. Agent cannot recover from partial tool failure

Situation: Production users report: agent cannot recover from partial tool failure.

Likely root causes: No compensation logic, missing retry policy, state machine too weak.

Investigation path: Define tool failure taxonomy, retries, rollback, human escalation, and persisted workflow checkpoints.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

28. RAG gives different answers for similar questions

Situation: Production users report: rag gives different answers for similar questions.

Likely root causes: Query rewrite instability, retrieval nondeterminism, context ordering, model randomness.

Investigation path: Pin settings, compare traces, normalize rewrites, stabilize ranking, and test paraphrase robustness.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

29. Security review blocks launch

Situation: Production users report: security review blocks launch.

Likely root causes: Prompt injection, sensitive data disclosure, excessive agency, insufficient audit logs.

Investigation path: Map risks to controls: permissions, sandboxing, allowlists, human approval, red-team tests, and incident logging.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

30. Support tickets increased after adding agent automation

Situation: Production users report: support tickets increased after adding agent automation.

Likely root causes: Agent resolves easy cases but mishandles edge cases or escalation.

Investigation path: Segment tickets by task type, inspect failed traces, add escalation thresholds, and measure human override rate.

Logs/metrics to inspect: Trace IDs, retrieved chunks, tool calls, filter expressions, prompt/model versions, latency, cost, user feedback, and deployment timeline.

Short-term mitigation: Rollback risky change, disable high-risk tool/action, widen observability, re-index stale data, or force citation/refusal mode.

Long-term fix: Add regression tests, pipeline validation, safer tool contracts, monitoring, and release gates.

Strong interview answer: I would reproduce with a trace, isolate the pipeline stage, prove the root cause with logs/metrics, mitigate safely, then add tests so it does not recur.

Weak interview answer: I would improve the prompt or switch to a better model.

Part 6 - Evaluation Framework Questions

Evaluation is where many AI interviews become senior-level. A strong candidate separates retrieval quality from generation quality, offline evaluation from online feedback, and model quality from product success. Evaluation must cover answer correctness, evidence support, citation accuracy, tool behavior, cost, latency, security, and customer-specific requirements.

1. How would you evaluate a RAG system before production?

Strong answer: Start with a representative golden set segmented by tenant, language, document type, and task. Measure retrieval recall/context precision, answer faithfulness, citation accuracy, refusal correctness, latency, and cost. Add human review for high-risk answers and regression tests for known incidents.

Weak answer: A weak answer mentions accuracy or user feedback without separating retrieval, generation, citations, security, cost, and task success.

What the interviewer is looking for: Evidence that you can design evaluation as an engineering system, not as a single metric.

Scorecard: 5/5: segmented metrics, test data, failure analysis, release gates, and monitoring. 3/5: basic metrics and some human review. 1/5: vague accuracy discussion.

2. How do you know retrieval is the problem?

Strong answer: Run answer evaluation with gold context and compare against production retrieval context. Measure recall@k and inspect whether the required evidence appears in top-k and final prompt. If the model answers correctly with gold context but fails with retrieved context, retrieval/context is likely the bottleneck.

Weak answer: A weak answer mentions accuracy or user feedback without separating retrieval, generation, citations, security, cost, and task success.

What the interviewer is looking for: Evidence that you can design evaluation as an engineering system, not as a single metric.

Scorecard: 5/5: segmented metrics, test data, failure analysis, release gates, and monitoring. 3/5: basic metrics and some human review. 1/5: vague accuracy discussion.

3. How do you measure hallucination?

Strong answer: Define hallucination as unsupported, contradicted, stale, or inaccessible claims. Use claim extraction, citation support checks, human review, and task-specific labels. Track hallucination rate by domain, source type, language, and prompt/model version.

Weak answer: A weak answer mentions accuracy or user feedback without separating retrieval, generation, citations, security, cost, and task success.

What the interviewer is looking for: Evidence that you can design evaluation as an engineering system, not as a single metric.

Scorecard: 5/5: segmented metrics, test data, failure analysis, release gates, and monitoring. 3/5: basic metrics and some human review. 1/5: vague accuracy discussion.

4. How do you evaluate an agent that uses tools?

Strong answer: Measure task success, tool selection accuracy, argument correctness, unnecessary tool calls, failure recovery, human escalation quality, safety policy compliance, latency, and cost per successful task.

Weak answer: A weak answer mentions accuracy or user feedback without separating retrieval, generation, citations, security, cost, and task success.

What the interviewer is looking for: Evidence that you can design evaluation as an engineering system, not as a single metric.

Scorecard: 5/5: segmented metrics, test data, failure analysis, release gates, and monitoring. 3/5: basic metrics and some human review. 1/5: vague accuracy discussion.

5. How do you build a regression test suite for AI systems?

Strong answer: Collect production incidents, edge cases, high-value workflows, security probes, and customer-specific examples. Freeze expected evidence and acceptable answer criteria. Run tests on prompt, model, retrieval, index, parser, and tool changes before release.

Weak answer: A weak answer mentions accuracy or user feedback without separating retrieval, generation, citations, security, cost, and task success.

What the interviewer is looking for: Evidence that you can design evaluation as an engineering system, not as a single metric.

Scorecard: 5/5: segmented metrics, test data, failure analysis, release gates, and monitoring. 3/5: basic metrics and some human review. 1/5: vague accuracy discussion.

6. How do you balance cost, latency, and quality?

Strong answer: Define quality thresholds by task risk. Use model routing, caching, smaller models for simple tasks, reranking only when needed, token budgets, and human approval for high-risk cases. Optimize cost per successful task, not token cost alone.

Weak answer: A weak answer mentions accuracy or user feedback without separating retrieval, generation, citations, security, cost, and task success.

What the interviewer is looking for: Evidence that you can design evaluation as an engineering system, not as a single metric.

Scorecard: 5/5: segmented metrics, test data, failure analysis, release gates, and monitoring. 3/5: basic metrics and some human review. 1/5: vague accuracy discussion.

7. How do you evaluate citation accuracy?

Strong answer: Check whether each cited source supports the exact claim, whether the cited span exists in the final context, and whether citations reflect the source actually used. Penalize generic document-level citations when the claim needs sentence-level evidence.

Weak answer: A weak answer mentions accuracy or user feedback without separating retrieval, generation, citations, security, cost, and task success.

What the interviewer is looking for: Evidence that you can design evaluation as an engineering system, not as a single metric.

Scorecard: 5/5: segmented metrics, test data, failure analysis, release gates, and monitoring. 3/5: basic metrics and some human review. 1/5: vague accuracy discussion.

8. How do you evaluate prompt injection defenses?

Strong answer: Use malicious documents, malicious user prompts, cross-tenant leakage tests, tool-abuse scenarios, and sensitive action attempts. Measure whether the system follows trusted instructions, ignores untrusted instructions, limits tool access, and logs security-relevant events.

Weak answer: A weak answer mentions accuracy or user feedback without separating retrieval, generation, citations, security, cost, and task success.

What the interviewer is looking for: Evidence that you can design evaluation as an engineering system, not as a single metric.

Scorecard: 5/5: segmented metrics, test data, failure analysis, release gates, and monitoring. 3/5: basic metrics and some human review. 1/5: vague accuracy discussion.

9. How do you evaluate multilingual RAG?

Strong answer: Create parallel and native-language test sets, evaluate retrieval recall by language, check cross-lingual retrieval, assess translated query rewriting, and involve native-speaking reviewers for high-risk domains.

Weak answer: A weak answer mentions accuracy or user feedback without separating retrieval, generation, citations, security, cost, and task success.

What the interviewer is looking for: Evidence that you can design evaluation as an engineering system, not as a single metric.

Scorecard: 5/5: segmented metrics, test data, failure analysis, release gates, and monitoring. 3/5: basic metrics and some human review. 1/5: vague accuracy discussion.

10. How do you evaluate freshness?

Strong answer: Measure time from source update to serving index availability. Add tests for effective-date policies, archive filters, document versioning, and stale-source refusal. Track freshness SLOs per corpus.

Weak answer: A weak answer mentions accuracy or user feedback without separating retrieval, generation, citations, security, cost, and task success.

What the interviewer is looking for: Evidence that you can design evaluation as an engineering system, not as a single metric.

Scorecard: 5/5: segmented metrics, test data, failure analysis, release gates, and monitoring. 3/5: basic metrics and some human review. 1/5: vague accuracy discussion.

Evaluation Metrics Map

Metric	What it measures / failure it catches
Retrieval recall@k	Whether needed evidence is retrieved; catches retriever misses, low top-k, and bad filters.
Context precision	How much provided context is useful; catches noisy context and false positives.
Faithfulness / groundedness	Whether claims are supported by context; catches unsupported synthesis.
Citation accuracy	Whether citations support exact claims; catches decorative or wrong citations.
Tool-call accuracy	Whether the agent selects the correct tool and arguments.
Task success rate	Whether the workflow actually completes the user goal.
Cost per successful task	Economic efficiency at target quality.
Human escalation quality	Whether uncertain/risky cases are routed safely.

Part 7 - Weak vs Strong Interview Answers

This section trains calibration. In interviews, a weak answer is often not factually false; it is incomplete, shallow, or unproven. A strong answer names the failure mode, the evidence needed, the debugging path, the mitigation, and the trade-off.

1. Why does a RAG system hallucinate?

Weak answer: Because the prompt is weak or the model is not good enough.

Why the weak answer is weak: It jumps to the final generation layer and ignores retrieval, context, data, and evaluation failures.

Strong answer: RAG hallucination can come from missing or stale documents, bad parsing, chunking, embeddings, filters, retrieval recall, reranking, context ordering, conflicting evidence, or weak grounding. I would inspect the trace to see whether the model saw the right evidence before changing the model.

Why the strong answer is strong: It treats hallucination as a pipeline failure, not just a model flaw.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

2. What do you investigate first when production answers become wrong?

Weak answer: I would improve the model or tune temperature.

Why the weak answer is weak: It suggests changes before reproducing and observing the failure.

Strong answer: I would reproduce a failing trace, compare it with a known-good trace, inspect retrieved chunks, source versions, metadata filters, context construction, prompt/model versions, and recent releases.

Why the strong answer is strong: It follows an evidence-driven investigation path.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

3. How do you debug retrieval quality?

Weak answer: I would increase top-k.

Why the weak answer is weak: Top-k may help but does not identify the root cause.

Strong answer: I would build labeled queries, measure recall@k and precision, inspect false positives/negatives, compare raw vs rewritten queries, dense vs lexical vs hybrid results, and evaluate reranking separately.

Why the strong answer is strong: It separates candidate generation, filtering, and ranking.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

4. How do you design an agent for customer support?

Weak answer: Use an LLM with tools connected to ticketing and knowledge base.

Why the weak answer is weak: It ignores permissions, escalation, observability, and action safety.

Strong answer: I would define read tools, write tools, risk levels, approval gates, ticket idempotency, retrieval grounding, escalation rules, audit logs, and task-success evaluation.

Why the strong answer is strong: It designs controlled autonomy.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

5. How do you prevent prompt injection?

Weak answer: Tell the model to ignore malicious prompts.

Why the weak answer is weak: Instruction-only defense is insufficient.

Strong answer: I would treat retrieved/user content as untrusted data, isolate system instructions, limit tool permissions, validate outputs, require approval for sensitive actions, red-team malicious documents, and log security events.

Why the strong answer is strong: It uses defense-in-depth.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

6. How do you evaluate RAG quality?

Weak answer: Ask users if answers are good.

Why the weak answer is weak: User feedback is useful but too slow and unstructured alone.

Strong answer: I would evaluate retrieval recall, context precision, faithfulness, answer relevance, citation accuracy, refusal correctness, latency, cost, and human review by segment.

Why the strong answer is strong: It defines a complete evaluation system.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

7. How do you reduce LLM latency?

Weak answer: Use a faster model.

Why the weak answer is weak: It ignores retrieval, reranking, token budget, streaming, and caching.

Strong answer: I would break down latency by retrieval, reranking, prompt construction, model generation, and tools. Then I would reduce tokens, cache, route models, batch reranking, stream output, and set latency budgets by task.

Why the strong answer is strong: It optimizes the whole pipeline.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

8. How do you handle stale documents?

Weak answer: Re-upload the documents.

Why the weak answer is weak: It lacks lifecycle and freshness controls.

Strong answer: I would track source modified time, parse time, embedding time, index commit time, effective date, expiry date, and serving freshness. Then I would automate re-indexing and stale-source filtering.

Why the strong answer is strong: It distinguishes document freshness from index freshness.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

9. How do you design tenant isolation?

Weak answer: Add tenant_id to the prompt.

Why the weak answer is weak: Prompt-level isolation is not security.

Strong answer: I would enforce tenant and ACL filters before retrieval, use tenant-aware namespaces or filters, include tenant in cache keys, test cross-tenant queries, and deny by default.

Why the strong answer is strong: It treats security as architecture.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

10. How do you decide between fine-tuning and RAG?

Weak answer: Fine-tuning is better for accuracy.

Why the weak answer is weak: It overgeneralizes.

Strong answer: I would use RAG for dynamic knowledge and provenance, fine-tuning for style, format, domain behavior, or repeated task patterns. For factual changing enterprise knowledge, I usually start with RAG plus evaluation before fine-tuning.

Why the strong answer is strong: It connects technique to business constraints.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

11. What is the difference between groundedness and correctness?

Weak answer: They are the same.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: A grounded answer is supported by given context; a correct answer matches reality. A response can be grounded in stale evidence and still be wrong, or correct but unsupported by context.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

12. How do you debug wrong citations?

Weak answer: Fix citation prompt.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would inspect whether citations are span-bound, whether the cited text supports the claim, and whether post-processing mismatched source IDs.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

13. How do you handle conflicting documents?

Weak answer: Use the highest ranked chunk.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would use source authority, effective dates, conflict detection, and either apply business rules or explicitly surface uncertainty.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

14. How do you evaluate an agent?

Weak answer: Check if the final answer looks good.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would evaluate task success, tool selection, arguments, recovery, cost, latency, safety, and human escalation.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

15. How do you stop an agent from looping?

Weak answer: Set a low max iteration count.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would add termination criteria, state checks, repeated-observation detection, planner constraints, and escalation when progress stalls.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

16. How do you make tool calls safe?

Weak answer: Trust the model to choose tools.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would use typed schemas, allowlists, permission scopes, dry-run mode, approval gates, idempotency keys, and audit logs.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

17. How do you debug multilingual failures?

Weak answer: Translate everything to English.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would evaluate language detection, multilingual embeddings, query rewriting, chunking, and native-language golden sets.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

18. How do you handle long context?

Weak answer: Use a larger context window.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would improve retrieval, reduce irrelevant context, order evidence, test lost-in-middle behavior, and keep critical evidence near the front.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

19. What is a good RAG observability trace?

Weak answer: Model input and output.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: It includes raw query, rewrite, filters, retrieved chunks, scores, reranks, context, prompt/model versions, citations, latency, cost, and feedback.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

20. How do you know a model upgrade is safe?

Weak answer: Run a few manual tests.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would run regression suites for answer quality, faithfulness, refusal, citations, tool behavior, latency, cost, and security probes.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

21. How do you build a golden dataset?

Weak answer: Write random questions.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would sample real production queries, high-risk workflows, known incidents, edge cases, multilingual cases, and expected evidence.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

22. How do you handle missing answer cases?

Weak answer: Force the model to answer.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would detect answerability, require citations, allow refusal, suggest where to look, and log gaps for corpus improvement.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

23. How do you measure business impact?

Weak answer: Accuracy score.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would measure resolved tasks, time saved, escalation rate, CSAT, cost per success, risk incidents, and adoption.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

24. How do you handle private data in RAG?

Weak answer: Tell the model not to reveal it.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would enforce ACLs before retrieval, minimize data, redact where needed, log access, and test leakage paths.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

25. How do you debug cost spikes?

Weak answer: Use a cheaper model.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would break down cost by model, tokens, retrieval, reranking, tools, retries, loops, and cache misses.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

26. How do you test prompt changes?

Weak answer: Deploy and see feedback.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would version prompts, run offline regression, canary rollout, monitor segments, and keep rollback available.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

27. How do you design fallback behavior?

Weak answer: Apologize on errors.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would classify failures, use safe fallback answers, retry where appropriate, escalate risky cases, and preserve audit trails.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

28. How do you handle numeric answers?

Weak answer: Ask model to calculate carefully.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would parse structured data, validate units, use deterministic calculators/tools, and cite source values.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

29. How do you deal with table-heavy documents?

Weak answer: Chunk them like text.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would use table-aware parsing, preserve headers, cell coordinates, captions, and test row/column queries.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

30. How do you handle API failures in agents?

Weak answer: Retry the call.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would use typed errors, exponential backoff, circuit breakers, idempotency, partial-state recovery, and user-visible status.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

31. How do you design agent memory?

Weak answer: Store conversation history.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would scope memory by user/task, define retention, permissions, update rules, expiry, and user control.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

32. How do you debug answer inconsistency?

Weak answer: Lower temperature.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would compare traces for query rewrite, retrieval set, ordering, prompt, model version, random seed/settings, and cache.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

33. How do you prove a fix worked?

Weak answer: The answer looks better.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would rerun failing cases, regression suites, segment metrics, and monitor production after rollout.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

34. How do you prioritize fixes?

Weak answer: Fix whatever is easiest.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would prioritize by user impact, risk, frequency, blast radius, and confidence in root cause.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

35. How do you explain trade-offs to stakeholders?

Weak answer: Give technical details.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would translate trade-offs into quality, latency, cost, risk, and user experience with options and recommendation.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

36. How do you prevent over-automation?

Weak answer: Let the agent do everything.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would classify actions by risk and require human approval for irreversible, financial, legal, or sensitive operations.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

37. How do you debug bad query rewriting?

Weak answer: Disable rewriting.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would compare raw/rewrite retrieval, preserve entities/negations, evaluate rewrite quality, and use fallback routing.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

38. How do you handle data drift?

Weak answer: Update the model.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would monitor query/document distribution changes, refresh evaluation sets, and segment quality over time.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

39. How do you design for auditability?

Weak answer: Log conversations.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would log decisions, evidence, tool calls, user approvals, policy checks, model/prompt versions, and final outputs.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

40. How do you answer when you are unsure in an interview?

Weak answer: Guess confidently.

Why the weak answer is weak: It is too shallow or unsafe for production work.

Strong answer: I would state assumptions, propose a diagnostic path, and identify what evidence would confirm or reject each hypothesis.

Why the strong answer is strong: It shows operational judgment and precise vocabulary.

Senior-level improvement: Add the exact metric, trace field, rollback option, and long-term regression test you would use.

Part 8 - Scorecards and Rubrics

Use these rubrics to evaluate your own answers. A premium interview answer is not just correct; it is structured, scoped, evidence-driven, and aware of operational risk.

RAG troubleshooting

Score	What the candidate demonstrates
1/5 - Poor	Gives generic answers for rag troubleshooting, relies on prompts/models, and misses production constraints.
2/5 - Basic	Names some components but cannot connect them to concrete failure modes or evidence.
3/5 - Good	Provides a reasonable architecture or debugging path with some metrics, but misses edge cases or trade-offs.
4/5 - Strong	Structures the answer, names risks, metrics, logs, mitigations, and explains why each step matters.
5/5 - Excellent	Demonstrates staff-level judgment: prioritization, blast radius, rollback, evaluation, security, observability, and customer impact.

RAG architecture design

Score	What the candidate demonstrates
1/5 - Poor	Gives generic answers for rag architecture design, relies on prompts/models, and misses production constraints.
2/5 - Basic	Names some components but cannot connect them to concrete failure modes or evidence.
3/5 - Good	Provides a reasonable architecture or debugging path with some metrics, but misses edge cases or trade-offs.
4/5 - Strong	Structures the answer, names risks, metrics, logs, mitigations, and explains why each step matters.
5/5 - Excellent	Demonstrates staff-level judgment: prioritization, blast radius, rollback, evaluation, security, observability, and customer impact.

Agent system design

Score	What the candidate demonstrates
1/5 - Poor	Gives generic answers for agent system design, relies on prompts/models, and misses production constraints.
2/5 - Basic	Names some components but cannot connect them to concrete failure modes or evidence.
3/5 - Good	Provides a reasonable architecture or debugging path with some metrics, but misses edge cases or trade-offs.
4/5 - Strong	Structures the answer, names risks, metrics, logs, mitigations, and explains why each step matters.
5/5 - Excellent	Demonstrates staff-level judgment: prioritization, blast radius, rollback, evaluation, security, observability, and customer impact.

Production debugging

Score	What the candidate demonstrates
1/5 - Poor	Gives generic answers for production debugging, relies on prompts/models, and misses production constraints.
2/5 - Basic	Names some components but cannot connect them to concrete failure modes or evidence.
3/5 - Good	Provides a reasonable architecture or debugging path with some metrics, but misses edge cases or trade-offs.
4/5 - Strong	Structures the answer, names risks, metrics, logs, mitigations, and explains why each step matters.
5/5 - Excellent	Demonstrates staff-level judgment: prioritization, blast radius, rollback, evaluation, security, observability, and customer impact.

Evaluation framework design

Score	What the candidate demonstrates
1/5 - Poor	Gives generic answers for evaluation framework design, relies on prompts/models, and misses production constraints.
2/5 - Basic	Names some components but cannot connect them to concrete failure modes or evidence.
3/5 - Good	Provides a reasonable architecture or debugging path with some metrics, but misses edge cases or trade-offs.
4/5 - Strong	Structures the answer, names risks, metrics, logs, mitigations, and explains why each step matters.
5/5 - Excellent	Demonstrates staff-level judgment: prioritization, blast radius, rollback, evaluation, security, observability, and customer impact.

Security and prompt injection

Score	What the candidate demonstrates
1/5 - Poor	Gives generic answers for security and prompt injection, relies on prompts/models, and misses production constraints.
2/5 - Basic	Names some components but cannot connect them to concrete failure modes or evidence.
3/5 - Good	Provides a reasonable architecture or debugging path with some metrics, but misses edge cases or trade-offs.
4/5 - Strong	Structures the answer, names risks, metrics, logs, mitigations, and explains why each step matters.
5/5 - Excellent	Demonstrates staff-level judgment: prioritization, blast radius, rollback, evaluation, security, observability, and customer impact.

Observability and monitoring

Score	What the candidate demonstrates
1/5 - Poor	Gives generic answers for observability and monitoring, relies on prompts/models, and misses production constraints.
2/5 - Basic	Names some components but cannot connect them to concrete failure modes or evidence.
3/5 - Good	Provides a reasonable architecture or debugging path with some metrics, but misses edge cases or trade-offs.
4/5 - Strong	Structures the answer, names risks, metrics, logs, mitigations, and explains why each step matters.
5/5 - Excellent	Demonstrates staff-level judgment: prioritization, blast radius, rollback, evaluation, security, observability, and customer impact.

Customer problem-solving

Score	What the candidate demonstrates
1/5 - Poor	Gives generic answers for customer problem-solving, relies on prompts/models, and misses production constraints.
2/5 - Basic	Names some components but cannot connect them to concrete failure modes or evidence.
3/5 - Good	Provides a reasonable architecture or debugging path with some metrics, but misses edge cases or trade-offs.
4/5 - Strong	Structures the answer, names risks, metrics, logs, mitigations, and explains why each step matters.
5/5 - Excellent	Demonstrates staff-level judgment: prioritization, blast radius, rollback, evaluation, security, observability, and customer impact.

Trade-off communication

Score	What the candidate demonstrates
1/5 - Poor	Gives generic answers for trade-off communication, relies on prompts/models, and misses production constraints.
2/5 - Basic	Names some components but cannot connect them to concrete failure modes or evidence.
3/5 - Good	Provides a reasonable architecture or debugging path with some metrics, but misses edge cases or trade-offs.
4/5 - Strong	Structures the answer, names risks, metrics, logs, mitigations, and explains why each step matters.
5/5 - Excellent	Demonstrates staff-level judgment: prioritization, blast radius, rollback, evaluation, security, observability, and customer impact.

Senior engineering judgment

Score	What the candidate demonstrates
1/5 - Poor	Gives generic answers for senior engineering judgment, relies on prompts/models, and misses production constraints.
2/5 - Basic	Names some components but cannot connect them to concrete failure modes or evidence.
3/5 - Good	Provides a reasonable architecture or debugging path with some metrics, but misses edge cases or trade-offs.
4/5 - Strong	Structures the answer, names risks, metrics, logs, mitigations, and explains why each step matters.
5/5 - Excellent	Demonstrates staff-level judgment: prioritization, blast radius, rollback, evaluation, security, observability, and customer impact.

Hiring Signal by Level

Level	Signal
Junior	Can explain basic RAG/agent concepts but needs guidance to debug production issues. Tends to focus on models and prompts.
Mid-level	Can build common RAG/agent features and debug obvious failures with logs, but may miss evaluation, security, and edge cases.
Senior	Can independently diagnose pipeline failures, design safe agent workflows, communicate trade-offs, and build evaluation/observability loops.
Staff-level	Can define platform standards, risk controls, org-wide evaluation strategy, incident response, cost governance, and customer-facing rollout plans.

Part 9 - Final Interview Preparation Playbook

The goal of preparation is not memorization. The goal is to build a reusable answer structure for any RAG, agent, evaluation, or production incident question.

If the Interview Is Tomorrow

- Memorize the production debugging loop: reproduce, classify, inspect retrieval, inspect context, inspect generation, inspect evaluation, mitigate, fix permanently.
- Practice five scenarios out loud: stale documents, retrieval miss, wrong citation, agent wrong tool, prompt injection.
- Prepare three phrases: "I would first inspect the trace," "retrieved is not the same as seen," and "grounded does not always mean correct if the source is stale."
- Review the scorecards and avoid weak answers such as "use a better model" or "improve the prompt" without evidence.

7-Day Preparation Plan

Day	Focus	Output
1	RAG failure modes	Explain 10 failure modes without notes.
2	Retrieval and context debugging	Practice tracing a query from user input to final prompt.
3	Agent system design	Design two agents with tools, state, approval, and evaluation.
4	Production scenarios	Answer 10 debugging scenarios out loud.
5	Evaluation frameworks	Define metrics for RAG, agents, security, cost, and latency.
6	Weak vs strong answers	Rewrite weak answers into senior answers.
7	Mock interview	Run a 60-minute mock with scorecard self-review.

14-Day Preparation Plan

Days	Focus	Output
1-2	RAG foundations through failure modes	Create your personal RAG troubleshooting map.
3-4	Chunking, embeddings, retrieval, reranking	Build answer templates for retrieval quality questions.
5-6	Context construction and grounded generation	Practice citation, stale-document, and conflict scenarios.
7-8	Agent design	Design five agent systems with safety controls.
9-10	Evaluation	Create a metric suite for RAG and agent systems.
11-12	Production debugging	Practice 20 scenarios with investigation paths.
13	Communication	Practice trade-offs: quality vs latency vs cost vs risk.
14	Mock interview and review	Score yourself using Part 8 rubrics.

30-Day Preparation Plan

Week	Focus	Output
1	RAG systems and failure modes	Write 25 short failure-mode explanations.
2	Agents, tools, state, security	Design 10 agent workflows and evaluate their risks.
3	Evaluation and production debugging	Build a sample evaluation plan and incident playbook.
4	Mock interviews and refinement	Complete 3 mocks, improve weak answers, and prepare portfolio stories.

What to Memorize

- RAG pipeline stages: query, ingestion, parsing, chunking, embedding, retrieval, reranking, context construction, generation, evaluation, monitoring.
- Debugging loop: reproduce, classify, trace, inspect evidence, isolate stage, mitigate, fix, test.
- Agent safety controls: tool contracts, permissions, approval gates, idempotency, retries, state, audit logs, evaluation.
- Evaluation dimensions: retrieval recall, context precision, groundedness, citation accuracy, task success, latency, cost, security.

What Not to Memorize

- Do not memorize vendor-specific buzzwords without understanding failure modes.
- Do not memorize one perfect architecture; interviews change constraints intentionally.
- Do not claim all hallucinations are model problems.
- Do not overstate certainty. Senior engineers state assumptions and tests.

How to Speak Like a Senior AI Engineer

- Use evidence language: trace, reproduce, compare, isolate, measure, segment, rollback, regression test.
- Use risk language: blast radius, tenant isolation, permission boundary, approval gate, audit trail, fallback.
- Use product language: user impact, task success, cost per successful task, support tickets, trust, adoption.
- Use trade-off language: quality vs latency, recall vs precision, autonomy vs safety, freshness vs cost.

Common Mistakes Candidates Make

- They blame the model before inspecting retrieval and context.
- They describe architecture but not observability.
- They design agents without permission boundaries.
- They treat citations as a UI feature instead of a faithfulness guarantee.
- They optimize for demo success instead of production reliability.
- They fail to separate retrieval evaluation from generation evaluation.
- They give confident answers without naming assumptions.

Appendix - Reference Anchors and Cheat Sheets

This book is a practical interview guide. The following public references are useful anchors for deeper study and for framing interview answers. They should not be treated as the only way to build production systems, but they help establish shared vocabulary.

Reference	Why it matters
Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, NeurIPS 2020	Original RAG framing: combining parametric model knowledge with non-parametric retrieved memory.
Liu et al., Lost in the Middle: How Language Models Use Long Contexts, TACL/arXiv 2023-2024	Important evidence that long context alone does not guarantee reliable use of middle-position evidence.
OWASP Top 10 for Large Language Model Applications / GenAI Security Project	Security framing for prompt injection, sensitive information disclosure, excessive agency, and other LLM application risks.
NIST AI Risk Management Framework: Generative AI Profile, 2024	Risk-management framing for organizations deploying generative AI systems.

One-Page RAG Debugging Cheat Sheet

- Start with a trace, not a theory.
- Ask: Did the right source enter the corpus? Was it parsed correctly? Was it chunked as usable evidence?
- Ask: Were embeddings and index versions compatible and fresh?
- Ask: Did retrieval find the right evidence? Did reranking keep it? Did filters remove it?
- Ask: Did the final prompt include the evidence in a usable position?
- Ask: Did the model answer only from context and cite the exact supporting span?
- Ask: Do evaluation sets represent current production queries?

One-Page Agent Design Cheat Sheet

- Define task boundaries before tools.
- Separate read-only tools from write/action tools.
- Use typed tool schemas and validate arguments.
- Add permission checks before tool execution.
- Require human approval for high-risk or irreversible actions.
- Persist state for long-running workflows and use idempotency keys.
- Log planner decisions, tool calls, observations, errors, approvals, and final outcomes.
- Evaluate task success, tool accuracy, safety, cost, latency, and escalation quality.